My Pocket – LLM Agent Report

Oct 28, 2026 ·

Course: Advanced AI & Software Engineering, C-Clarke International Institute of Digital Sciences (Online Group 02)

Group: Sanjula Nawodya, Chamod Mihiranga, Komul Harana

Repository: github.com/Nawodyaaaaaa/My-Pocket

1. Problem and scenario

My Pocket is a personal finance agent that tracks expenses in Sri Lankan rupees (LKR) and answers spending questions, running on a local LLM through Ollama.

Many young people in Sri Lanka do not know where their money goes each month. Spending is spread across small daily items such as canteen lunches, bus fares, PickMe rides, mobile reloads and course fees. Budget apps need forms, and they cannot answer a simple question like “Am I over budget on food this month?”

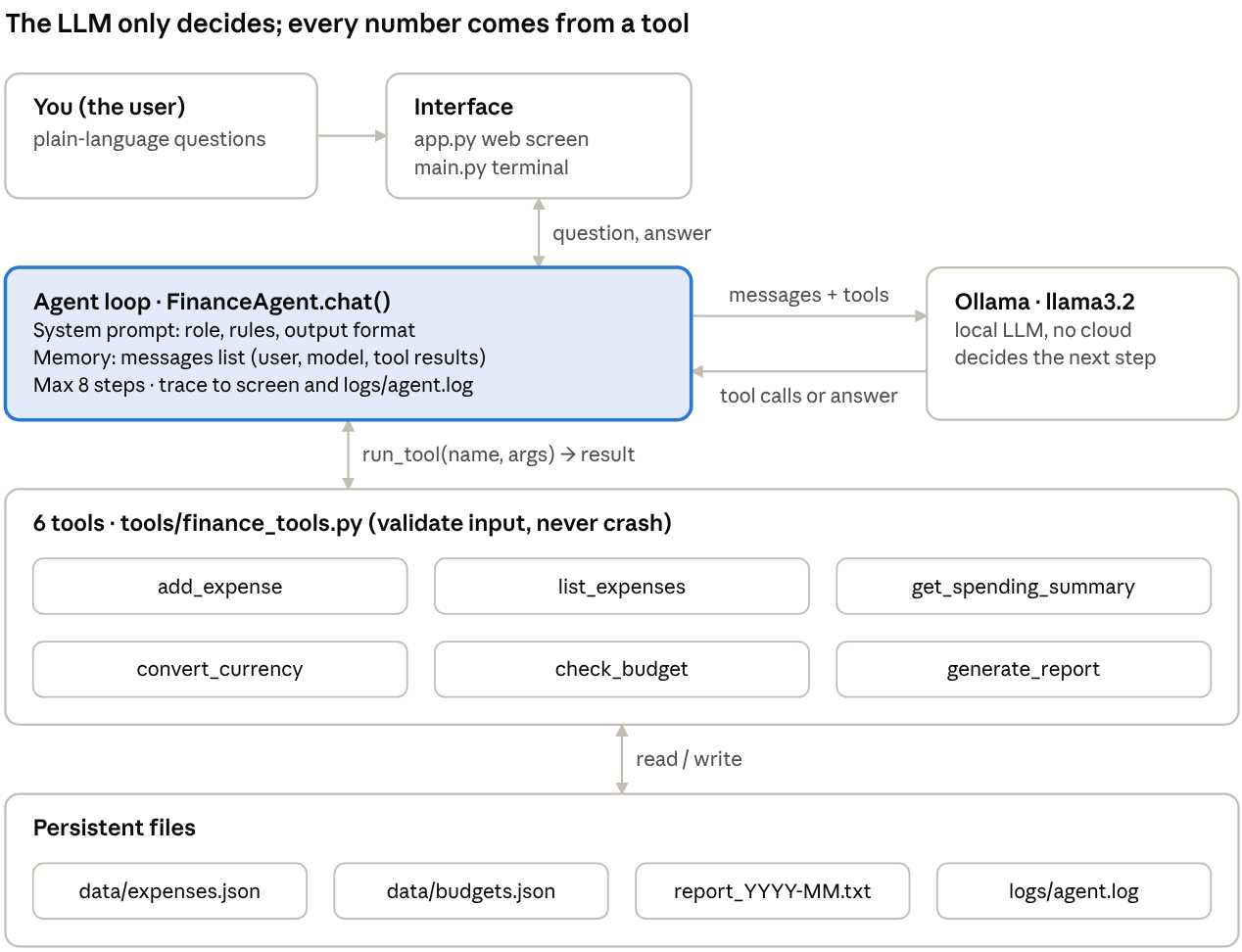
We chose the **Personal finance helper** scenario and made it a local variation:

- All amounts are in **LKR** (“Rs. 12,500”), with conversion to USD, EUR, GBP, INR, AUD and JPY.
- Categories fit daily life: food, transport, bills, shopping, entertainment, health, education and other.
- The user types in plain language (“I spent 1500 rupees on lunch today”) instead of filling in a form.

The agent decides which tools to call and in what order. For example, *“How much did I spend on food this month in USD, and am I over budget?”* needs three tools: a spending summary, a currency conversion and a budget check.

2. Architecture

The LLM decides what to do, the tools do the work, and the agent loop passes messages between them until there is a final answer.



My Pocket architecture · interface, agent loop, local LLM, 6 tools, files

A question goes from the interface to the agent loop, which sends the conversation and the tool list to llama3.2. The model replies with tool calls or a final answer. Tool results go back into memory and the loop repeats.

3. Model choice and prompt design

We use **llama3.2 (3B, about 2 GB)** through Ollama. It supports tool calling and runs on a normal laptop without a GPU. Larger models such as mistral 7B are often better at multi-step tasks, but they are slower on our hardware.

The app does not hard-code the model. `pick_model()` asks Ollama which models are installed and picks one that supports tools, so nobody had to download a new model.

System prompt

The system prompt has three parts:

1. **Role:** "You are My Pocket, a personal finance assistant for a user in Sri Lanka." It also includes today's date, so "this month" works.
2. **Rules:** always use tools for numbers and never guess; chain tools for multi-step questions; if a tool returns an error, explain it; ask if the request is unclear; never make payments.
3. **Output format:** short answers, amounts as "Rs. 12,500", bullet points for breakdowns, and one tip when the user is over budget.

Design decisions

- **"Never guess numbers"** pushes the model to call a tool instead of inventing amounts.
- **Tool descriptions matter.** Each tool in `tools/schemas.py` has a one-line purpose and typed parameters. Categories are a fixed list, so the model can only pick a valid one.
- **After testing** we added a rule never to write tool calls as text, and set temperature to 0 for steadier choices.

4. Tool design

The agent has six tools in `tools/finance_tools.py`. Each one checks its inputs and never crashes: bad input returns `{"ok": false, "error": "..."}`, which the model reads and explains.

Tool	Purpose	Parameters
`add_expense`	Save an expense in LKR to `data/expenses.json`	amount, category, note, date_str (empty = today)
`list_expenses`	List a month's expenses, newest first	month, category, limit
`get_spending_summary`	Month total and totals per category	month, category
`convert_currency`	Convert between LKR and USD, EUR, GBP, INR, AUD, JPY (fixed mock rates)	amount, from_currency, to_currency

Tool	Purpose	Parameters
`check_budget`	Compare spending with limits in `data/budgets.json`	month, category
`generate_report`	Write `reports/report_YYYY-MM.txt`	month

Sample outputs (October 2026 sample data)

`get_spending_summary(month="2026-10", category="food")`

```
{"ok": true, "month": "2026-10", "category": "food", "total_lkr": 30440.0,
"number_of_expenses": 18}
```

`convert_currency(amount=30440, from_currency="LKR", to_currency="USD")`

```
{"ok": true, "result": 101.47, "rate_used": "1 USD = 300.0000 LKR", "note":
"Mock rates for demo purposes, not live market data."}
```

`check_budget(month="2026-10", category="food")`

```
{"ok": true, "results": [{"category": "food", "budget_lkr": 30000,
"spent_lkr": 30440.0, "percent_used": 101.5, "status": "OVER budget"}]}
```

`add_expense(amount=-500, category="food", note="pizza")`

```
{"ok": false, "error": "Amount must be a positive number, got '-500'."}
```

Data is stored in JSON files, so it survives a restart. Exchange rates are fixed (1 USD = Rs. 300) and every result says they are mock rates.

5. Agent loop, memory and logging

The loop is plain Python with the `ollama` library, in `FinanceAgent.chat()` (`agent/finance_agent.py`). No framework is used, so every step is easy to explain.

```
self.messages.append({"role": "user", "content": user_input})

for step in range(1, MAX_STEPS + 1):          # MAX_STEPS = 8
    response = ollama.chat(model=self.model, messages=self.messages,
                           tools=TOOL_SCHEMAS, options={"temperature": 0})
    msg = response.message
    self.messages.append(msg)

    calls = tool calls from the reply (or ones the model typed as text)
    if not calls:                              # no tool needed = final
answer
        return msg.content

    for name, args in calls:
        result = self.run_tool(name, args)     # never crashes
        log.info(...)                          # trace: tool, arguments,
result
        self.messages.append({"role": "tool", "tool_name": name,
                              "content": json.dumps(result)})
```

How it works. Each round sends the conversation and the tool list to the model. If the model asks for tools, we run them, add the results to the conversation and go round again. When it replies without a tool call, that reply is the final answer.

- **Step limit:** 8 rounds at most, so a confused model cannot loop forever.
- **Safe tools:** `run_tool()` turns an unknown tool or wrong arguments into an error message instead of a crash.
- **Memory:** `self.messages` keeps the whole conversation, so follow-ups like “What about in euros?” work. “Clear chat memory” resets it.
- **Logging:** every tool call and result is printed, saved to `logs/agent.log`, and shown in the web screen.
- **Text tool calls:** small models sometimes type a tool call as text. `find_text_tool_calls()` finds it and runs it anyway.

Interfaces: a web screen (`app.py`, Streamlit) with a chat box, example buttons, a tool-call trace under each answer and a budget sidebar; a terminal version (`main.py`); and `START.bat` for one-click start on Windows.

6. Testing

We have **16 automated tests, and all pass.** They need no Ollama and use a temporary copy of the data.

- `tests/test_tools.py` (13 tests): saving expenses, rejecting bad amounts, categories and months, currency conversion (USD, EUR, GBP), budget checks, listing expenses newest first, and the report file.
- `tests/test_agent.py` (3 tests): a fake model checks that the loop runs a tool call the model typed as text, and ignores unknown tools.

Real runs with llama3.2

#	Request	Result
1	How much did I spend this month, and what's my biggest category?	Worked: called `get_spending_summary`, gave the total and food as the biggest category
2	How much did I spend on food this month in USD, and am I over budget?	Partly: made 3 tool calls (`convert_currency`, `check_budget` twice) but misread the result and said food was within budget
3	What about in euros?	Worked: used the food total from memory and converted it (about €121)
4	I spent 1500 rupees on lunch today. Add it, then make this month's report.	Worked: expense saved and report written
5	Add an expense of -500 for pizza	Partly: the tool rejected it, but the model described it as income

What failed and how we fixed it

- **Tool call written as text.** In the first run of task 2, llama3.2 typed a tool call as text and skipped `check_budget`. We added a parser that runs such calls, a stricter prompt and temperature 0.
- **A test used live data.** Demo runs changed the food total, so a test failed. All tests now use a temporary copy of the data.

- **No time to download a model.** `pick_model()` uses a tool-capable model that is already installed.
- **Demo data changed after practice runs.** We restore the clean `expenses.json` before recording.

7. Limitations and future improvements

Limitations

- **Small model.** llama3.2 sometimes misreads tool results or skips a step, as in tasks 2 and 5.
- **Mock exchange rates**, not live data.
- **JSON storage** is fine for one user, not for many.
- **No login**, and no way to edit or delete an expense yet.
- **Slow first answer** on a CPU-only laptop while the model loads.

Future improvements

1. Try a stronger local model such as qwen3 and compare accuracy.
2. Move storage to SQLite.
3. Use a live exchange-rate API.
4. Add edit and delete tools, with a confirmation step.
5. Add spending charts to the web screen.

8. Individual contribution

Member	Role	Main work
Sanjula Nawodya	Agent lead	Agent loop, prompt, web screen, repository, demo video, report
Chamod Mihiranga	Tools	Fixed the agent tests to use temporary data (pull request #1)
Komul Harana	Testing	Added 3 tests: EUR and GBP conversion, and list_expenses order

Appendix: use of AI tools

As allowed by the assignment guide, we used Claude (Anthropic) as a coding assistant.

- Claude helped write the first version of the code and tests, explained errors and setup steps, and helped draft this report.
- We ran and tested the agent with a real model, found and fixed problems, made our own changes, and checked the report.

Libraries: Ollama and the ollama Python library, Streamlit, pytest.